

Cours 4 — Erreurs et bugs · La logique binaire

Semaine 4 ■ Manuel Hachette ch. 2 (§ 2.6) et ch. 9 (§ 9.1) ■ 2×1 h

Objectifs

- Reconnaître les trois familles d'erreurs et lire un message d'erreur Python
- Connaître le bit, les booléens et les opérateurs ET, OU, NON
- Construire une table de vérité

Partie 1 — Erreurs et « bugs » (cours A)

1.1 Trois familles d'erreurs

Famille	Quand ?	Exemples
Erreur de syntaxe	avant l'exécution : le code est mal écrit	: oublié, parenthèse non fermée → <code>SyntaxError</code> , <code>IndentationError</code>
Erreur d'exécution	pendant l'exécution : une opération impossible	<code>NameError</code> (variable inconnue), <code>TypeError</code> (types incompatibles), <code>ZeroDivisionError</code> , <code>ValueError</code> (<code>int("abc")</code>)
Erreur de logique	le programme tourne... mais le résultat est faux	mauvais ordre de tests, < au lieu de <=, mauvaise formule

Les deux premières se **voient** (Python proteste); la troisième est la plus dangereuse : seuls des **tests** la révèlent.

1.2 Lire un message d'erreur (traceback)

```
Traceback (most recent call last):
  File "script.py", line 4, in <module>
    print(prix + " euros")
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Lire **de bas en haut** : le type d'erreur et son explication (dernière ligne), puis la ligne de code concernée. Réflexe : aller voir la ligne indiquée *et celle du dessus*.

1.3 Débusquer un bug de logique

- Tester sur des **cas simples dont on connaît la réponse** (et les cas limites : 0, valeurs égales, extrêmes).
- Afficher les valeurs intermédiaires avec des `print` temporaires, ou exécuter pas à pas avec le **débogueur** de l'éditeur.
- Expliquer son code à quelqu'un d'autre, ligne à ligne (même à un objet : c'est la méthode du « canard en plastique »!).

Partie 2 — La logique binaire (cours B)

2.1 Le bit

Toute l'information d'un ordinateur (nombres, textes, images, sons, programmes) est représentée par des **bits** : des 0 et des 1 (*binary digit*). Physiquement : tension électrique, aimantation, creux microscopiques d'un DVD... Un groupe de 8 bits est un **octet** (*byte*).

2.2 Les opérateurs logiques

Un bit s'interprète comme un booléen : 1 = **True** = vrai, 0 = **False** = faux. Trois opérations fondamentales, données par leur **table de vérité** :

<i>a</i>	<i>b</i>	<i>a</i> ET <i>b</i>	<i>a</i> OU <i>b</i>	NON <i>a</i>
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

- **ET** (**and**) : vrai seulement si **les deux** sont vrais.
- **OU** (**or**) : vrai si **au moins un** est vrai (OU inclusif!).
- **NON** (**not**) : inverse.

On rencontre aussi le **OU exclusif** (XOR) : vrai si *a* et *b* sont **différents**.

2.3 Pourquoi c'est fondamental

- Ce sont les briques des **circuits** du processeur (portes logiques — revu au ch. 1).
- Ce sont les conditions de nos programmes : `age >= 15 and age < 18`.
- Lois utiles (à vérifier par table de vérité) : $\text{NON}(a \text{ ET } b) = (\text{NON } a) \text{ OU } (\text{NON } b)$ — lois de De Morgan.

À retenir

- Syntaxe / exécution / logique : trois familles d'erreurs, la troisième ne se voit qu'en testant.
- Un traceback se lit de bas en haut ; la ligne fautive est indiquée.
- 1 octet = 8 bits ; tout est bits dans une machine.
- Tables de vérité de ET, OU, NON par cœur ; OU est inclusif.